

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Narmatha J

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Deadlock Debugging

C Program (Deadlock Detection & Fix)

```
#include <stdio.h>

int main() {
    int R1 = 1, R2 = 1;
    int P1_R1 = 0, P2_R2 = 0;
    int P1_wait = 0, P2_wait = 0;
    // P1 locks R1
    if(R1) {
        P1_R1 = 1;
        R1 = 0; }
    // P2 locks R2
    if(R2) {
        P2_R2 = 1;
        R2 = 0;}
    // Both request another resource
    if(!R2)
        P1_wait = 1;
    if(!R1)
        P2_wait = 1;
    if(P1_wait && P2_wait)
        printf("Deadlock Detected\n");
    else
        printf("No Deadlock\n");
    return 0;
}
```

Output:

Deadlock Detected!

Optimization:

Lock resources in the same order.

Both processes should lock R1 first and then R2.

Q2. CPU Utilisation Debugging**C Program**

```
#include <stdio.h>

int main() {

    char state[] = {'R','W','R','W','R','W','R','W','R','W'};

    int ready = 0, waiting = 0;

    int n = sizeof(state)/sizeof(state[0]);

    for(int i=0;i<n;i++)

    {

        if(state[i]=='R')

            ready++;

        else

            waiting++;

    }

    float cpu = (ready*100.0)/(ready+waiting);

    printf("CPU Utilisation = %.2f%%\n",cpu);
```

```
if(cpu<50)

    printf("Problem Detected\n");

    return 0;

}
```

Output:

CPU Utilisation = 50.00%

Problem Detected

Debug Result:

Processes are spending most of the time waiting for I/O.

Optimization:

- Use asynchronous I/O and better CPU scheduling.
- Increase CPU burst execution before blocking.

Q3. Memory Bottleneck Debugging

C Program

```
#include <stdio.h>

int main()

{

    int code = 5;

    int heap = 150;

    int threads = 25;

    int stack = 1;
```

```
int total = code + heap + (threads * stack);

printf("Memory Usage\n");

printf("Code : %d MB\n", code);

printf("Heap : %d MB\n", heap);

printf("Stack : %d MB\n", threads * stack);

printf("\nTotal Memory = %d MB\n", total);

printf("\nProblem Detected:\n");

printf("Frequent page faults due to large heap allocation and many threads.\n");

printf("\nOptimization 1:\n");

printf("Reduce heap allocation.\n");

printf("\nOptimization 2:\n");

printf("Reduce number of threads or use thread pool.\n");

return 0;}
```

Output:

Memory Usage

Code : 5 MB

Heap :150 MB

Stack :25 MB

Total Memory =180 MB

Problem Detected:

Frequent page faults due to large heap allocation and many threads.

Optimization 1:

Reduce heap allocation.

Optimization 2:

Reduce number of threads or use thread pool.

Q4. Starvation Debugging**C Program**

```
#include <stdio.h>

int main()
{
    printf("Queue Status\n");

    printf("Q1 : Interactive Processes\n");

    printf("Q2 : System Processes\n");

    printf("Q3 : Batch Process P5\n");

    printf("\nObservation:\n");

    printf("P5 always waits because higher priority queues never become empty.\n");

    printf("\nRoot Cause:\n");

    printf("Starvation in Multilevel Queue Scheduling.\n");

    printf("\nOptimization:\n");

    printf("Apply Aging Technique.\n");

    printf("Increase priority of waiting processes over time.\n");

    return 0;}
```

Output:

Queue Status

Q1 : Interactive Processes

Q2 : System Processes

Q3 : Batch Process P5

Observation:

P5 always waits because higher priority queues never become empty.

Root Cause:

Starvation in Multilevel Queue Scheduling.

Optimization:

Apply Aging Technique.

Increase priority of waiting processes over time.

Q5. Context Switch Overhead Debugging**C Program**

```
#include <stdio.h>

int main() {
    int burst = 2;
    int quantum = 1;
    int switches = burst / quantum;
    printf("Context Switches per Thread = %d\n",switches);
    if(quantum < burst)
        printf("High Context Switch Overhead\n");
}
```

```
    return 0;  
}
```

Output:

Context Switches per Thread = 2

High Context Switch Overhead

Debug Result:

Very frequent preemption causes excessive context switching.

Optimization 1:

Increase time quantum.

Optimization 2:

Reduce unnecessary preemption and improve I/O scheduling.

